Übungsaufgaben Gesamtsicht

SIEMENS

Kapitel 1: Konzept und Architektur der Virtual-Threads

- V-T 1: Virtual-Threads starten und auf sie warten — Einfach
- V-T 2: Blocking-Call gibt Carrier-Thread frei — Mittel
- V-T 16: Fibonacci parallelisieren zeigt Ressourcenvorteil der Virtual-Threads — Schwer

Kapitel 2: Technisches Fundament, Blocking, Realesing

- V-T 6: Lock.lock Blocking Verhalten — Einfach
- V-T 8: Queue: Blocking-Verhalten — Mittel
- V-T 17 Architecture Speedup — Schwer

Kapitel 3: Perfomance und Skalierbarkeit

- V-T 3: Skalierbarkeit bei Blocking-Calls — Einfach
- V-T 4: Performance bei Blocking-Calls — Mittel
- V-T 15: Server-Skalierbarkeit — Schwer

Kapitel 4: Structured Concurrency, ScopedValues

- S-C 01 Scope based Structured Concurrency — Einfach
- S-C 02 Sub-Scopes — Mittel
- S-C 03 Joiner — Schwer
- S-V 00 Scoped Values Einfaches Beispiel — Einfach
- S-V 01 Scoped Values Basic API — Mittel
- S-V 02 Scoped Values Threading — Mittel

Kapitel 5: Anwendungsarchitektur und Technologieauswahl

- V-T 10 ForkJoinTasks vs. Virtual-Threads — Mittel
- V-T 17 Architecture Speedup — Schwer

Kapitel 6: Sonderaspekte und Probleme der Virtual-Threads

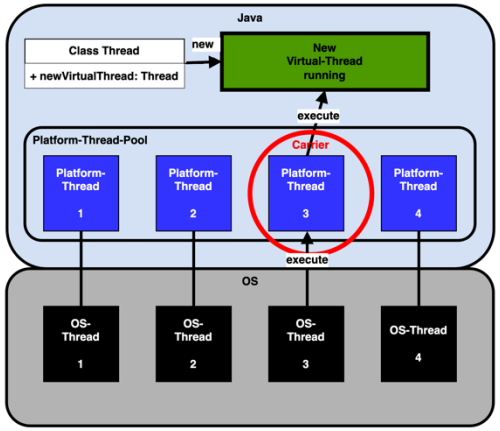
- V-T 13: Preemtion und Programmsemantik — Mittel
- VT 14: Thread-Pool Konfiguration — Mittel

© Marwan Abu-Khalil

Übung V-T 1
Virtual-Threads starten und auf sie warten

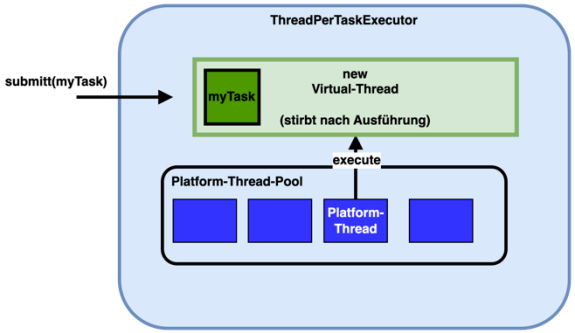
Einfach

SIEMENS



1. **Starte einen einzelnen Virtual-Thread**
   - Nutze eine statische Methode der Klasse Thread, um einen Virtual-Thread zu starten
   - Schreibe aus dem Virtual-Thread heraus dessen ID auf die Konsole
   - Warte auf das Ende des Virtual-Threads

2. **Nutze einen Virtual-Thread mithilfe eines ExecutorService**
   - Verwende Executors.newVirtualThreadPerTaskExecutor(), um einen geeigneten ExecutorService zu erhalten
   - Starte im ExecutorService einen Virtual-Thread und schreibe dessen String- Repräsentation auf die Konsole
   - Warte auf das Ende der Ausführung des Threads, nutze dafür ein Future Objekt



- **Lernziel**: Virtual-Threads starten, auf sie warten und sie identifizieren können
- **Musterlösung**: src/exercises/**vt_01_api**

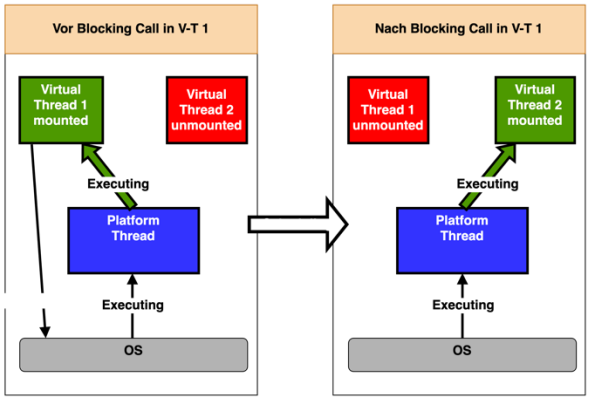
© Marwan Abu-Khalil

Übung V-T 2
Blocking-Call gibt Carrier-Thread frei

Mittel

SIEMENS

Zeige, dass ein Virtual-Thread, der einen blockierenden Aufruf ausführt, den unterliegenden Platform-Thread („Carrier“) freigibt



- Nutze blockierenden Aufruf: Thread.sleep()
- Identifiziere Carrier-Thread durch: Thread.currentThread()
- Zeige, dass ein Virtual-Thread verschiedene Carrier haben kann, indem Du den Carrier vor und nach sleep() vergleichst

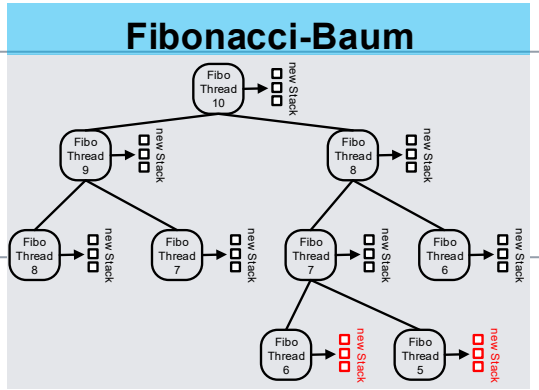
```
Virtual Thread ID and underlying Carrier ID before sleep: VirtualThread[#26]/runnable@ForkJoinPool-1-worker-1
Virtual Thread ID and underlying Carrier ID before sleep: VirtualThread[#28]/runnable@ForkJoinPool-1-worker-2
Virtual Thread ID and underlying Carrier ID before sleep: VirtualThread[#30]/runnable@ForkJoinPool-1-worker-3
Virtual Thread ID and underlying Carrier ID before sleep: VirtualThread[#34]/runnable@ForkJoinPool-1-worker-4
Virtual Thread id and underlying Carrier ID after sleep: VirtualThread[#34]/runnable@ForkJoinPool-1-worker-2
Virtual Thread id and underlying Carrier ID after sleep: VirtualThread[#26]/runnable@ForkJoinPool-1-worker-1
Virtual Thread id and underlying Carrier ID after sleep: VirtualThread[#28]/runnable@ForkJoinPool-1-worker-4
Virtual Thread id and underlying Carrier ID after sleep: VirtualThread[#30]/runnable@ForkJoinPool-1-worker-3
```

- **Lernziel**: Verstehen, dass blockierende Calls aus Virtual-Threads den unterliegenden Platform-Thread nicht blockieren
- **Musterlösung**: src/exercises/**vt_02_blocking**

© Marwan Abu-Khalil

Übung V-T 16
Fibonacci parallelisieren zeigt Ressourcenvorteil der Virtual-Threads

Schwer



Parallelisiere die rekursive Berechnung einer Fibonacci Zahl.

- **Scenario 1:** Eine naive Parallelisierung mit Platform-Threads stürzt ab, weil sie zu viel Speicher verbraucht (Naiv heißt: In jeder Rekursiosstufe zwei neue Threads starten).
  - Hier erkennen, dass die Platfrom-Threads sehr “teuer” sind

- **Scenario 2**: Eine naive Parallelisierung mit Virtual-Threads ist hingegen stabil aber langsam
  - **Dies ist das wesentliche Lernergebnis**

- Scenario 3: Eine Parallelisierung mit einem sogenannten Threshold ist effizient.
  - (Der Threshold begrenzt, bis zu welcher Rekursionstiefe neue Threads gestartet werden)
  - (Es gibt dabei kaum Unterschied zwischen Platform-Threads und Virtual-Threads)

- **Lernziel**: Erkennen, dass Platform Threads wesentlich teurer sind als Virtual-Threads
- **Musterlösung**: src/exercises/**vt_16_fibonacci_virt_plat**

© Marwan Abu-Khalil

Übung V-T 3
Skalierbarkeit bei Blocking

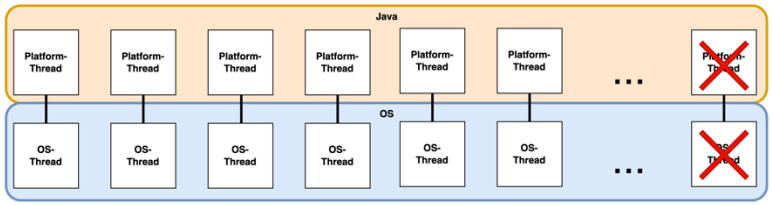
Einfach

SIEMENS

Zeige, dass eine Anwendung mit Virtual-Threads besser skalieren kann als mit Platform-Threads, wenn die Threads blockierende Aufgaben haben

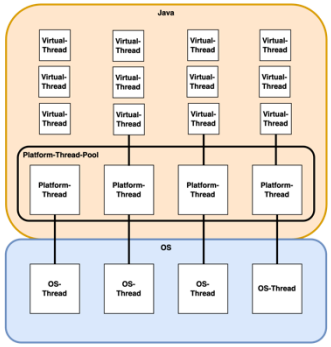
1. Starte so viele Platform Threads, dass das Programm abstürzt
   - Nutze dafür z.B. Thread.sleep() um die Threads blockierend am Leben zu erhalten

```
Thread thread = new Thread(()->{
  // …
});
thread.start();
```



2. Zeige, dass wesentlich mehr virtuelle Threads gleichzeitig existieren können

```
Thread virtualThread =
  Thread.startVirtualThread(() -> {
    // …
  }
);
```



3. Erkläre den Unterschied

Lernziel: Erkennen, dass Virtual-Threads das Skalierbarkeitsproblem der Platform-Threads lösen

Musterlösung: rc/exercises/**vt_03_scalability_blkg**

© Marwan Abu-Khalil

Mittel

SIEMENS

Übung V-T 4
Performance durch Virtual-Threads bei Blocking-Calls

Finde einen Use Case, bei dem die parallele Ausführung mit Virtual-Threads schneller ist, als die parallele Ausführung mit Platform-Threads

1. (Ist in der Klasse Performance_BASE bereits implementiert)
   - Definiere eine Aufgabe, die mit einem blockierenden OS-Aufruf arbeitet
   - Starte so viele dieser Aufgaben mit Platform-Threads, dass das Programm noch stabil läuft
   - Messe die Laufzeit

2. Zeige, dass dieses Programm mit virtuellen Threads beschleunigt werden kann, da mehr virtuelle Threads gestartet werden können als Platform Threads.

3. Erkläre wie der Performance Vorteil zustande kommt

Lernziel: Verstehen, dass der Hauptnutzen der Virtual-Threads in der verbesserten Performance von Server-Applikationen liegt

Musterlösung: src/exercises/**vt_04_performance_blkg**

© Marwan Abu-Khalil

Übung V-T 6
Lock.lock() Blocking-Verhalten

Einfach

SIEMENS

Zeige, dass die Methode lock() der expliziten Lock Klassen aus java.util.concurrent den Carrier Thread freigibt, wenn lock() in einem virtuellen Thread aufgerufen wird.

Programmiere folgendes Szenario:

- Starte viele (mehr als Grad der Parallelität des Pools) virtuelle Threads, die eine Funktion aufrufen, die eine Zeit lang auf der CPU läuft, z.B. eine Sekunde
- Zeige, dass nicht alle Virtual-Threads sofort starten, sondern nur so viele, wie es Platform Threads im Pool gibt
- Baue dann eine sogenannte Critical Section mit ReantrantLock.lock() und ReentrantLock.unlock() in die langlaufende Funktion ein
- Zeige, dass nun weitere virtuelle Threads starten, sobald die vorhergehenden virtuellen Threads die lock() Methode aufrufen.

Lernziel: Blocking Verhalten Virtual-Threads am Beispiel von Lock kennenlernen

Musterlösung: src/exercises/**vt_06_lock**

Console

```
<terminated> Lock_BASE [Java Application] /Library/Java/JavaVirtualMachines/jdk-25.jdk/Contents/Home/bin/java  (25 Oct 2025, 21:26:30
virtual thread running 11 at 6 Milliseconds VirtualThread[#50]/runnable@ForkJoinPool-1-worker-12
virtual thread running 5 at 5 Milliseconds VirtualThread[#37]/runnable@ForkJoinPool-1-worker-5
virtual thread running 8 at 6 Milliseconds VirtualThread[#43]/runnable@ForkJoinPool-1-worker-10
virtual thread running 3 at 5 Milliseconds VirtualThread[#34]/runnable@ForkJoinPool-1-worker-4
virtual thread running 14 at 642 Milliseconds VirtualThread[#54]/runnable@ForkJoinPool-1-worker-8
virtual thread running 15 at 642 Milliseconds VirtualThread[#55]/runnable@ForkJoinPool-1-worker-6
```

Zwei Threads starten später!

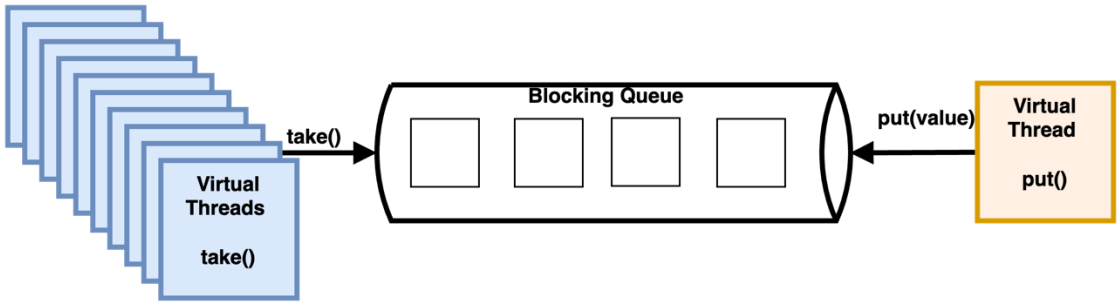
Übung V-T 8
Queue: Blocking-Verhalten

Mittel

SIEMENS

Zeige, dass ein Virtual-Thread, der einen blockierenden Aufruf einer Queue benutzt, im Falle des Blockierens seinen unterliegenden Carrier Thread freigibt

- Nutze dafür eine `java.util.concurrent.BlockingQueue`
- Starte viele virtuelle Threads, die queue.take() aufrufen (mehr als Pool-Threads vorhanden sind)
- Diese take()-threads blockieren, da die Queue leer ist.
- Starte dann einige virtuelle Threads, die mit queue.put() einen Wert in die Queue einfügen.
- Zeige, dass es zu keinem Deadlock kommt, Obwohl alle diese virtuellen Threads einen gemeinsamen Threadpool beschränkter Groesse benutzen.



Lernziel: Verhalten von Virtual-Threads am Beispiel von Producer-Consumer Szenarien verstehen

Musterlösung: src/exercises/**vt_09_queue_blocking**

© Marwan Abu-Khalil

V-T 10 ForkJoinTasks vs. Virtual-Threads

Mittel

SIEMENS

Zeige, dass ForkJoinTasks anders mit blockierenden Aufrufen umgehen als Virtual-Threads, da ForkJoinTasks Ihren unterliegenden Platform-Thread nicht freigeben.

Baue dafür ein Szenario, bei dem ForkJoinTasks in eine blockierende Queue Elemente mit put() einfügen und mit take() Elemente entnehmen. Dies beides sind blockierende Aufrufe.

A) Zeige, dass dabei mit ForkJoinTasks ein Deadlock entstehen kann (Hinweis: Viele ForkJoinTasks starten)

B) Zeige, dass sich dieses Problem mit Virtual-Threads lösen lässt

Lernziel: Unterschied zwischen Virtual-Threads und ForkJoinTasks erkennen

Musterlösung: JavaVirtualThr src/exercises/**vt_10_forkjoin**

© Marwan Abu-Khalil

Übung V-T 13
Preemption und Programmsemantik

Mittel

SIEMENS

Bei Virtual-Threads gibt es keine Preemption, das heißt, ein Virtual-Thread wird nicht von einem anderen Virtua-Thread verdrängt. Der laufende Thread muss von sich aus die Kontrolle abgeben, indem er endet oder blockiert.

Zeige, dass daher das Ergebnis eines Programmes ein anderes sein kann, je nachdem ob es mit Platform-Threads oder mit Virtual-Threads parallelisiert ist.

Lösungshinweis:

- Nutze Virtual-Threads die unendlich lange laufen, z.B. einen Zähler unendlich oft inkrementieren
- Starte dann P+2 solche Threads, wobei P die Parallelität im Threadpool ist
- Zeige, dass das Verhalten mit Platform Threads anders ist

Default Einstellung für P ist Anzahl der logischen CPUs

```
P = Runtime.getRuntime().availableProcessors();
```

Lernziel: Verstehen, dass die Migration von Platform-Threads zu Virtual-Threads die Semantik eines Programmes verändern kann

Musterlösung: src/exercises/**vt_13_preemption_semantics**

© Marwan Abu-Khalil

Übung V-T 14
Threadpool Konfiguration

Schwer

SIEMENS

Zeige, wie der Threadpool konfiguriert werden kann, und wie sich das auf das Verhalten der Virtual-Threads auswirkt

VM Parameter mit denen der Threadpool konfiguriert werden kann:

- -Djdk.virtualThreadScheduler.parallelism=X
- -Djdk.virtualThreadScheduler.maxPoolSize=Y
- -Djdk.virtualThreadScheduler.minRunnable=Z

A) Konfiguriere den Threadpool so, dass nur ein Virtual-Thread gleichzeitig laufen kann und zeige an einem Beispiel, dass dann andere Virtual-Threads warten müssen

B) Konfiguriere den Pool so, dass im Falle vieler pinning Threads der Pool auf 100 anwächst. Zeige dass damit auch bei Pinning 100 Virtual-Threads starten.

Lernziel: Erkennen, dass der unterliegende Threadpool das Verhalten der Virtual-Threads beinflusst

Musterlösung: src/exercises/**vt_14_threadpool_config**

© Marwan Abu-Khalil

Übung V-T 15
Server Skalierbarkeit

Schwer

SIEMENS

Baue einen einfachen Server, der mit Platform Threads bei vielen Anfragen in kurzer Zeit abstürzt. Verbessere ihn durch den Einsatz von Virtual-Threads.

Die Basis der Musterlösung impelmentiert bereits folgendes Szenario. Repariere es durch Einsatz von Virtual-Threads im Server

- Der Server startet für jeden Client-Request einen eigenen Thread, der den Request engegennimmt und eine Antwort zurück sendet.
- Der Client setzt viele (z.B. 10.000) Requests ab, ohne dazwischen zu warten. So entsteht Last auf dem Server
- Der Server stürzt ab einer gewissen Anzahl Requests ab (OutOfMemoryErrror bei der Thread-Erzeugung)

Lernziel: Erkennen, dass Server mit Virutal Threads besser skalieren

Musterlösung: src/exercises/**vt_15_client_server_scalability**

© Marwan Abu-Khalil

V-T 17 Architecture Speedup

Schwer

SIEMENS

Welche Architekturen und Use-Cases lassen sich mit Virtual Threads beschleunigen?

In der Basis zur Aufgabe, sind zwei Methoden mit unterschiedlichen Szenarien vorgegeben. Das eine ist CPU-intensiv, das andere ist I/O-bound. Beide sind mit Platform-Threads bereits parallelisiert.

A) Zeige, dass sich das eine Szenario durch Virtual Threads massiv beschleunigen lässt.

B) Erkläre, warum sich das andere nicht durch Virtual Threads beschleunigen lässt

Lernziel: Verstehen, welche Architekturen sich mit Virtual Threads beschleunigen lassen

Musterlösung: src/exercises/**vt_17_architecture_speedup**

(Bemerkung: Diese Aufgabe ist nicht wirklich schwer, aber an dieser Stelle im Seminar muss man noch ein bisschen nachdenken, um sie zu verstehen. Im Kapitel 5 kommt sie erneut)

© Marwan Abu-Khalil

Übung S-C 01
Scope based Structured Concurrency

Einfach

SIEMENS

Grundlagen des Arbeitens mit StructuredTaskScope

1. Scope aufbauen: Definiere einen Scope mithilfe der Klasse StructuredTaskScope

2. Subtasks erzeugen: Starte in diesem Scope zwei Subtasks unter Verwendung der Methode StructuredTaskScope.fork() und zeige, dass der Scope erst dann beendet wird, wenn beide Subtasks abgeschlossen sind (join())

3. Exception Handling: Werfe aus der ersten Subtasks eine Exception. Sorge dafuer, dass die zweite Subtask dadurch beendet wird.

Hinweise

- scope.join(); wartet auf alle Subtasks
- Jede Subtask muss ihren Interrupted Zustand regelmaessig pruefen: Thread.currentThread().isInterrupted()

Lernziel: Benutzung von SturcutedTaskScope erlernen und das Exceptionhandling darin verstehen

Musterlösung: src/exercises/struct_conc/**sc_01_structured_scope**

© Marwan Abu-Khalil

Übung S-C 02
Sub-Scopes

Mittel

SIEMENS

Es gibt mehrere Varianten von Sub-Scopes in einem StructuredTaskScope.

- Variante 1 Sub Scope innerhalb des umschließenden Scopes
- Variante 2 Sub-Scope innerhalb eines Subtasks des umschließenden Scopes

A) Variante 1 im Scope: Baue einen aeusseren Scope unter Verwendung von StructuredTaskScope.open. Erweitere den aeusseren Scope um einen Subscope (Variante 1). Starte einen Task-1 in diesem Subscope.

B) Variante 2 im Subtask: Erweitere das Ergebnis von A) indem Du in dem äusseren Scope einen Task-2 forkst, und erzeuge innerhalb dieses Task-2 einen Subscope. Darin startest Du dann wiederum eine Task-3.

C) Nebenläufigkeit: Wie lange müsste das Programm laufen, wenn Task-1 4 Sekunden, Task-2 3 Sekunden und Task-3 2 Sekunden lang laufen?

Lernziel: Varianten von Sub-Scopes kennenlernen. Nebenläufigkeit von Scopes verstehen.

Musterlösung: src/exercises/struct_conc/**sc_02_sub_scopes**

© Marwan Abu-Khalil

Übung S-C 03
Joiner

Schwer

SIEMENS

Joiner definieren das Verhalten eines StructuredTaskScope. Es gibt merhere vordefinierte Joiner, Man kann aber auch selbst defienerte Joiner schreiben.

- **A)** Baue einen Scope, der drei unterschiedlich lang laufende Subtasks umfasst.
  - **1)** Nutze einen Joiner, der dazu fuehrt, dass das Ergebnis derjenigen Subtask, die als erste fertig ist, zurueckgegeben wird. (Hinweis: `anySuccessfulResultOrThrow`)
  - **2)** Stelle sicher, dass sich die anderen Tasks beenden.

- **B)**
  - **1)** Baue einen eigenen Joiner, indem du das Interface StructuredTaskScope.Joiner<T, R> implementierst. Dieser soll das Maximum der Ergebnisse der Subtasks berechnen (Integer).
  - **2)** Baue einen Scope, der mehrere Tasks hat, die Integer Werte zurueckgeben. Benutze darin den Joiner aus 1) und zeige, dass er das Maximum der Rueckgabwerte findet.

Lernziel: Verhalten und Impelemntierung von Joinern verstehen

Musterlösung: src/exercises/struct_conc/**sc_03_joiners**

© Marwan Abu-Khalil

Übung S-V 00
Scoped Values Einfaches Beispiel

Einfach

SIEMENS

Aufgabe
Zeige, dass ein einzige ScopedValue Instanz in verschiedenen Virtual-Threads gleichzeitig unterschiedliche Werte haben kann

Hinweis:
- Eine ScopedValue Instanz anlegen, die von allen beteiligten Threads genutzt werden kann
- Mit ScopedValue.where(…) der ScopedValue Instanz einen Thread-Spezifischen Wert zuweisen
- Mit ScopedValue.where(…).**run(() -> {…})** den innerhalb des Scopes auszuführenden Code übergeben

Lernziel: Verstehen dass ein ScopedValue in jedem Thread einen eigenen spezifischen Wert hat

Musterlösung: src/exercises/scoped_values/sv_00_simple_example

© Marwan Abu-Khalil

Übung S-V 01
Scoped Values Basic API

Einfach

SIEMENS

Diese Aufgabe demonstriert das grundlegende API von Scoped Values: where() und run().

Es geht dabei um das Binden von Werten innerhalb von Scopes und den Zugriff auf diese Werte. Alles findet im Main Thread statt(!).

A) Binding und Scope

- Weise eine Variable vom Typ ScopedValue mit where() einen Wert zu
- Führe mit run() ein Lambda im diesem Scope aus
- Greife aus dem Lambda auf den Wert zu und schreibe ihn auf die Console.

B) Chainig

- Instanziiere mehrere Variablen vom Typ Scoped Value und weise ihnen mit verkettete where() Aufrufe Werte zu
- Starte mit run() ein Lambda, das auf diese Werte zugreifen kann, schreibe sie auf die Console

C) Rebinding

- Eröffne innerhalb eine Scoped-Value Scopes mit where() einen Sub-Scope
- Binde im Sub-Scope einen neuen Wert an den Scoped-Value.
- Zeige, dass innerhalb des Sub-Scopes der neue Wert gitl, und dass nach dem Verlassen des Sub-Scopes wieder der alte Wert gueltig ist.

Lernziel: Scoped-Values API kennen: Scope, Binding, Rebinding, Chaining

Musterlösung: src/exercises/scoped_values/**sv_01_basic_api**

© Marwan Abu-Khalil

Übung S-V 02
Scoped Values Threading

Mittel

SIEMENS

Der Sinn von Scoped Values ist es, diese in Threads zur Verfügung zu stellen. Dafuer gibt es unterschiedliche Herangehensweisen.

A) Inheritance identischer Werte im StructuredTaskScope
Nutze eine ScopedValue Instanz um mehrerern Threads einen indentischen Wert zur Verfuegung zu stellen. Verwende dafuer einen StructuredTaskScope.

B) Unterschiedliche Werte im StructuredTaskScope
Weise mehreren Threads innerhalb eines StructuredTaskScope jeweils unterschiedliche Werte zu

C) Unterschiedliche Werte in unstrukturierten Threads
Nutze Bindings (where()) in unstrukturierten Threads um jedem einen eigenen Wert zuzuweisen
.

Lernziel: Scoped-Values Verahlten bei Multithreading verstehen
Musterlösung: rc/exercises/scoped_values/**sv_02_threading**

© Marwan Abu-Khalil

SIEMENS

Literatur und Weblinks

- **Brian Goetz**, 2022: New Foundations for Java High Scale Applications https://www.infoq.com/articles/java-Virtual Threads/
- JEP Virtual Threads https://openjdk.org/jeps/425 and https://openjdk.org/jeps/444
- **JEP Structured Concurrency**: JEP 525 (Java 26, preview) https://openjdk.org/jeps/525
- **JEP 506: Scoped Values**, https://openjdk.org/jeps/506
- **Lutz Hühnken**, entwickler.de 2020 https://entwickler.de/java/blick-in-die-fernere-zukunft
- **Marwan Abu-Khalil**, 2025, Virtual Threads in Java: Performance und Skalierbarkeit in neuer Dimension, https://www.sigs.de/artikel/Virtual Threads-in-java-performance-und-skalierbarkeit-in-neuer-dimension/
- **Michael Inden**, 2025: Java 25 LTS: Modernes Java leicht gemacht https://www.amazon.de/Java-25-LTS-Modernes-gemacht/dp/B0FRYWGDJ7
- **Nicolai Parlog**, Oracle, 2022: https://blogs.oracle.com/javamagazine/post/java-loom-Virtual Threads-platform-threads
- **Oracle** Java Core Libraries Documentation: https://docs.oracle.com/en/java/javase/21/core/Virtual Threads.html
- **Reinald Menge-Sonntag**, heise online 2022: https://www.heise.de/news/Java-19-verbessert-die-Nebenlaeufigkeit-mit-virtuellen-Threads-aus-Project-Loom-7269453.html
- **Ricardo Cardin**, 2023, good Examples about Threadpool and Scheduling: https://blog.rockthejvm.com/ultimate-guide-to-java-Virtual Threads/#3-how-to-create-a-Virtual Thread
- **Ron Presler**, QCon 2019: https://www.infoq.com/presentations/continuations-java/
- **Sven Woltmann**, 2022, nice little examples: https://www.happycoders.eu/de/java/Virtual Threads/
- **Thomasz Nurkiewitz**, QCon 2022: https://www.infoq.com/presentations/loom-java-concurrency/

© Marwan Abu-Khalil

BAK

© Marwan Abu-Khalil

Technologieauswahl nach Programmierpattern und Architektur-Stil

SIEMENS

	Virtual-Thread	Platform-Thread	Fork-Join-Task	Parallel-Stream	Reactive-Stream
Rekursiver Algorithmus	**Möglich** Aber nicht primärer Use-Case (kein Blocking)	**Eher nicht** Vorsicht: Memory-Bedarf	**Ja** Optimal	**Nein** Rekursion nicht innerhalb Pipeline realisierbar	**Eher nicht** Fokus: verteiltes nebenläufiges System
Iterativer Algorithmus z.B. Matrix-Mult	**Möglich** Aber nicht primärer Use-Case (kein Blocking)	**Eher nicht** Vorsicht: Memory-Bedarf	**Ja** Optimal	**Ja** Wenn als eine Pipeline darstellbar	**Ja** Wenn viele Pipelines oder asynchrone Stufen
Producer-Consumer	**Ja** Insbesondere bei Blocking-Queue	**Ja** Garantierte Parallelität	**Nein** Blocking, keine garantierte Para.	**Nein** Pipeline nach 1. Durchlauf beendet	**Ja** Perfekter Use-Case
System-Architektur Strukturieren	**Ja** Aber nur im Fall skalierbarer Server	**Ja** Komponenten entkoppeln	**Nein** Kein Blocking kein IO	**Nein** Eher Design-Ebene	**Ja** Viele Subscriber Back-Pressure
UI-Thread entkoppeln	**Eher Nein** Platform-Thread besser	**Ja** Klassische Anwendung	**Ja** Kurzlaufende Hintergrund-Task	**Eher Nein** Asynchronität nicht im Fokus	**Ja** Typischer Use-Case z.B. in JS

© Marwan Abu-Khalil

Technologie-Auswahl nach funktionalen Eigenschaften (Capabilities der API)

SIEMENS

	Virtual-Thread	Platform-Thread	Fork-Join-Task	Parallel-Stream	Reactive-Stream	Completable-Future
Wenige / endliche Daten	**Ja** Billiger Kurzläufer	**Eher nicht** Langläufer	**Ja** Kurzläufer	**Ja** Effizient partitionierbar	**Evtl. Ja** Asynchronität mit Push, sonst C-F	**Ja** C-F kurzlebig
Viele Daten, unendliche Streams	**Ja** Wenn mit Blocking-I/O	**Ja** Langläufer	**Nein** Kurzläufer	**Nein** Nicht effizient partitionierbar	**Ja** perfekter Use-Case	**Nein** C-F läuft nur ein Mal
Pipeline-Verarbeitung	**Eher nicht** Keine API-Unterstützung	**Eher nicht** Keine API-Unterstützung	**Nein** Kein Blocking	**Ja** Perfekter Use-Case	**Ja** Keine Pipeline-Parallelität	**Nein** jedes C-F läuft nur ein Mal
Task-Parallelität	**Evtl. Ja** Falls keine Langläufer	**Ja** Unterschiedliche Aufgaben	**Evtl. Ja** Falls keine Langläufer	**Nein** Daten identisch verarbeitet	**Ja** Asynchrone Entkopplung	**Ja** Perfekter Use-Case
Daten-Parallelität	**Eher nicht** Keine API-Unterstützung	**Eher nicht** Kein API, Memory!	**Evtl. Ja** Wie Thread, aber billiger	**Ja** Perfekter Use-Case	**Eher nicht** Subscription sequentiell	**Eher nicht** Asynchron, nicht primät parallel
Blocking-I/O	**Ja** Perfekter Use-Case	**Ja**	**Nein** Blockiert Pool Thread	**Nein** Blockiert Pool Thread	**Ja** Asynchronität	**Ja** Asynchronität

Technologie-Auswahl nach nicht-funktionalen Anforderungen (operational und developmental)

SIEMENS

	Virtual-Thread	Platform-Thread	Fork-Join-Task	Parallel-Stream	Reactive-Stream	Completable-Future
Performance durch Parallelität	**Möglich** Aber nicht ideal	**Ja** Memory begrenzen	**Ja** kein autom. Threshold	**Ja** Perfekter Use-Case	**Eher nicht** Fokus Asynchronität	**Nein** Asynchronität, Entkopplung
Performance durch Asynchronität	**Ja**	**Ja**	**Eher Nein** Langläufer, Synch.	**Nein** Asynchronität nicht im Fokus	**Ja** Verteilt, nebenläufig	**Ja** Entkopplung von Aufgaben
Skalierbarkeit (automatisch)	**Ja**	**Nein**	**Ja** Threadpool	**Ja** Pool und Threshold	**Ja** Back-Pre. expilzit	**Nein**
Autom. Optimierung (Threshold)	**Nein**	**Nein**	**Nein**	**Ja**	**Nein**	**Nicht anwendbar**
Asynchrone APIs Design	**Ja**	**Nein**	**Nein**	**Nein**	**Ja** DB, Netzwerk,	**Ja** Perf. Use-Case
Transparent / Wartbar / Implizit parallel	**0**	**--** Explizit	**+** automatische Skalierbarkeit	**++** automatische Parallelität	**+** Parallel expl. deklariert	**+** Parallel expl. deklariert
Entkopplung / Modularisierbarkeit	**--**	**--**	**--**	**0**	**++**	**+**

© Marwan Abu-Khalil

SIEMENS

Vielen Dank!

© Marwan Abu-Khalil